

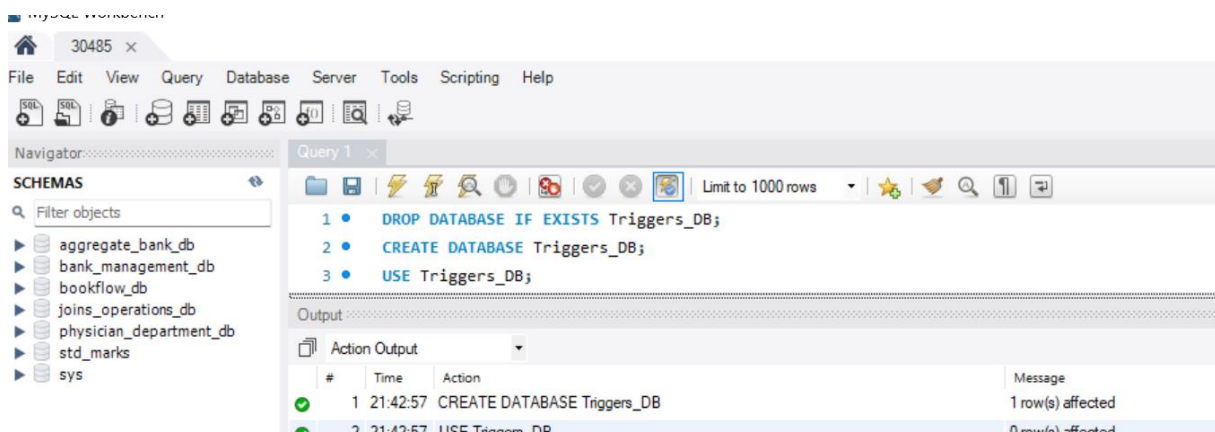
Triggers & Stored Procedures

M. Geethika (2520030485)

Question :

Develop a Bank Database using Customer, Account and Bank_Transaction tables. Implement stored procedures for customer and account operations, deposits, withdrawals, money transfer and balance retrieval. Implement BEFORE and AFTER triggers for balance validation, transaction validation, transaction auditing, automatic balance updates and insufficient-balance prevention.

Step 1 : Create Database and Table



MySQL Workbench

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Administration Schemas

Information

No object selected

Query 1

```

1 DROP DATABASE IF EXISTS Triggers_DB;
2 CREATE DATABASE Triggers_DB;
3 USE Triggers_DB;
4 -- Step 2: Create Customer Table
5 CREATE TABLE Customer (
6     Customer_ID INT PRIMARY KEY,
7     Customer_Name VARCHAR(100) NOT NULL,
8     Phone VARCHAR(15),
9     Email VARCHAR(100),
10    City VARCHAR(50)
11 );

```

Output

Action Output

#	Time	Action	Message
4	21:44:51	DROP DATABASE IF EXISTS Triggers_DB	0 row(s) affected
5	21:44:52	CREATE DATABASE Triggers_DB	1 row(s) affected
6	21:44:52	USE Triggers_DB	0 row(s) affected
7	21:44:52	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected

Object Info Session

MySQL Workbench

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Administration Schemas

Information

No object selected

Query 1

```

13 -- Step 3: Create Account Table
14
15 CREATE TABLE Account (
16     Account_No INT PRIMARY KEY,
17     Customer_ID INT,
18     Account_Type VARCHAR(20),
19     Balance DECIMAL(12,2) DEFAULT 0,
20     Branch VARCHAR(50),
21
22     FOREIGN KEY (Customer_ID)
23     REFERENCES Customer(Customer_ID)
24 );

```

Output

Action Output

#	Time	Action	Message
3	21:43:26	CREATE DATABASE Triggers_DB	Error Code: 1007. Can't create database tr
4	21:44:51	DROP DATABASE IF EXISTS Triggers_DB	0 row(s) affected
5	21:44:52	CREATE DATABASE Triggers_DB	1 row(s) affected
6	21:44:52	USE Triggers_DB	0 row(s) affected
7	21:44:52	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
8	21:47:16	DROP DATABASE IF EXISTS Triggers_DB	1 row(s) affected
9	21:47:16	CREATE DATABASE Triggers_DB	1 row(s) affected
10	21:47:16	USE Triggers_DB	0 row(s) affected
11	21:47:16	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
12	21:47:16	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID IN...	0 row(s) affected

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x

Limit to 1000 rows

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

26 -- Step 4: Create Bank Transaction Table

27

28 CREATE TABLE Bank_Transaction (

29 Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,

30 Account_No INT,

31 Transaction_Type VARCHAR(20),

32 Amount DECIMAL(12,2),

33 Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,

34

35 FOREIGN KEY (Account_No)

36 REFERENCES Account(Account_No)

37);

Output

Action Output

#	Time	Action	Message
9	21:47:16	CREATE DATABASE Triggers_DB	1 row(s) affected
10	21:47:16	USE Triggers_DB	0 row(s) affected
11	21:47:16	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
12	21:47:16	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID IN...	0 row(s) affected
13	21:47:51	DROP DATABASE IF EXISTS Triggers_DB	2 row(s) affected
14	21:47:51	CREATE DATABASE Triggers_DB	1 row(s) affected
15	21:47:51	USE Triggers_DB	0 row(s) affected
16	21:47:51	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
17	21:47:51	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID IN...	0 row(s) affected
18	21:47:51	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO...	0 row(s) affected

Object Info Session

No object selected

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x

Limit to 1000 rows

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

40 INSERT INTO Customer VALUES

41 (101, 'Geethika', '9876543210', 'harshika@gmail.com', 'Hyderabad'),

42 (102, 'Siri', '9876543211', 'ravi@gmail.com', 'Vijayawada'),

43 (103, 'Shailu', '9876543212', 'santhvin@gmail.com', 'Bangalore'),

44 (104, 'Jhansi', '9876543213', 'srilatha@gmail.com', 'Chennai'),

45 (105, 'Aishu', '9876543214', 'varshika@gmail.com', 'Hyderabad');

46

47 -- Step 6: Insert Account Data

48 INSERT INTO Account VALUES

49 (10001, 101, 'Savings', 50000, 'Hyderabad'),

50 (10002, 102, 'Savings', 75000, 'Vijayawada'),

51 (10003, 103, 'Current', 120000, 'Bangalore'),

52 (10004, 104, 'Savings', 45000, 'Chennai'),

53 (10005, 105, 'Current', 90000, 'Hyderabad');

54

Output

Action Output

#	Time	Action	Message
11	21:47:16	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
12	21:47:16	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID IN...	0 row(s) affected
13	21:47:51	DROP DATABASE IF EXISTS Triggers_DB	2 row(s) affected
14	21:47:51	CREATE DATABASE Triggers_DB	1 row(s) affected
15	21:47:51	USE Triggers_DB	0 row(s) affected
16	21:47:51	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
17	21:47:51	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID IN...	0 row(s) affected
18	21:47:51	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO...	0 row(s) affected

Object Info Session

No object selected

The screenshot displays the SQL Developer environment. The top menu bar includes File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The Navigator pane on the left shows the 'SCHEMAS' section with a list of databases: aggregate_bank_db, bank_management_db, bookflow_db, joins_operations_db, physician_department_db, std_marks, and sys. The main SQL window, titled 'Query 1', contains the following SQL code:

```
-- Step 7: Insert Transaction Data
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000);
```

The bottom pane shows the 'Output' window with the 'Action Output' tab selected. It displays the execution results of the SQL script:

#	Time	Action	Message
21	21:50:09	USE Triggers_DB	0 row(s) affected
22	21:50:09	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VA...	0 row(s) affected
23	21:50:09	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID IN...	0 row(s) affected
24	21:50:09	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO...	0 row(s) affected
25	21:50:09	INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALU...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
26	21:50:09	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Bran...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
27	21:51:49	DROP DATABASE IF EXISTS Triggers_DB	3 row(s) affected
28	21:51:49	CREATE DATABASE Triggers_DB	1 row(s) affected

30485 x
File Edit View Query Database Server Tools Scripting Help

SCHEMAS
Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Query 1 x
Limit to 1000 rows

```

63 (10005, 'WITHDRAW', 10000);
64
65
66 -- Step 8: Display Table Data
67 SELECT * FROM Customer;
68 SELECT * FROM Account;
69 SELECT * FROM Bank_Transaction;
70

```

SQLAdditions
Automat Use the help for or to

Result Grid
Filter Rows:
Edit:
Export/Import:
Wrap Cell Content:

	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
▶	1	10001	DEPOSIT	10000.00	2026-08-28 22:00:58
	2	10002	DEPOSIT	15000.00	2026-08-28 22:00:58
	3	10003	WITHDRAW	20000.00	2026-08-28 22:00:58
	4	10004	DEPOSIT	5000.00	2026-08-28 22:00:58
	5	10005	WITHDRAW	10000.00	2026-08-28 22:00:58

Administration Schemas

Customer 1 Account 2 Bank_Transaction 3 x
Aoolv Revert Context Help

Output
Action Output

#	Time	Action	Message
✓	40 22:00:58	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID I...	0 row(s) affected
✓	41 22:00:58	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUT...	0 row(s) affected
✓	42 22:00:58	INSERT INTO Customer VALUES (101, 'Geethika', '9876543210', hanahika@gmail...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓	43 22:00:58	INSERT INTO Account VALUES (10001, 101, 'Savings', 50000, 'Hyderabad'), (10...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓	44 22:00:58	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VAL...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓	45 22:00:58	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
✓	46 22:00:58	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
✓	47 22:00:58	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned

Object Info Session

Step 2 : STORED PROCEDURES

The screenshot displays the SQL Developer environment. The left sidebar shows a tree view of schemas, including 'aggregate_bank_db', 'bank_management_db', 'bookflow_db', 'joins_operations_db', 'physician_department_db', 'std_marks', and 'sys'. The main window is titled 'Query 1' and contains the following SQL code:

```
71 -- Step 9: Procedure to Display All Customers
72 DELIMITER //
73 CREATE PROCEDURE GetAllCustomers()
74 BEGIN
75     SELECT * FROM Customer;
76 END //
77 DELIMITER ;
78 -- Execute Step 9
79 CALL GetAllCustomers();
```

Below the code editor, the 'Result Grid' shows the output of the procedure, displaying a table with 5 rows and 5 columns:

Customer_ID	Customer_Name	Phone	Email	City
101	Geethika	9876543210	harshika@gmail.com	Hyderabad
102	Siri	9876543211	ravi@gmail.com	Vijayawada
103	Shailu	9876543212	santhvin@gmail.com	Bangalore
104	Jhansi	9876543213	sriatha@gmail.com	Chennai
105	Aishu	9876543214	varshika@gmail.com	Hyderabad

At the bottom, the 'Output' window shows the execution log:

#	Time	Action	Message
56	22:01:45	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VAL...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
57	22:01:45	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned
58	22:01:45	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
59	22:01:45	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
60	22:01:45	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; ...	0 row(s) affected
61	22:01:45	-- Execute Step 9 CALL GetAllCustomers()	5 row(s) returned

30485 X

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Query 1 x

```

85 )
86 BEGIN
87     SELECT *
88     FROM Account
89     WHERE Account_No = p_Account_No;
90 END //
91 DELIMITER ;
92 -- Execute Step 10
93 CALL GetAccountDetails(10001);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [F1](#)

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

Administration Schemas

Information

No object selected

Object Info Session

Customer 8 Account 9 Bank_Transaction 10 Result 11 Result 12 x

Output

Action Output

#	Time	Action	Message
72	22:03:10	SELECT * FROM Account LIMIT 0, 1000	5 row(s) returned
73	22:03:10	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
74	22:03:10	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; ...	0 row(s) affected
75	22:03:10	- Execute Step 9 CALL GetAllCustomers()	5 row(s) returned
76	22:03:10	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN ...	0 row(s) affected
77	22:03:10	- Execute Step 10 CALL GetAccountDetails(10001)	1 row(s) returned

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Query 1 x

Limit to 1000 rows

```

108 FROM Customer C
109 JOIN Account A
110     ON C.Customer_ID = A.Customer_ID
111 WHERE C.Customer_ID = p_Customer_ID;
112 END //
113 DELIMITER ;
114 -- Execute Step 11
115 CALL GetCustomerAccounts(101);
116

```

Result Grid

Filter Rows: Export: Wrap Cell Content: [F1](#)

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Geethika	10001	Savings	50000.00	Hyderabad

Administration Schemas

Information

No object selected

Object Info Session

Customer 13 Account 14 Bank_Transaction 15 Result 16 Result 17 Result 18 x

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
90	22:04:22	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; ...	0 row(s) affected
91	22:04:22	-- Execute Step 9 CALL GetAllCustomers()	5 row(s) returned
92	22:04:22	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN ...	0 row(s) affected
93	22:04:22	-- Execute Step 10 CALL GetAccountDetails(10001)	1 row(s) returned
94	22:04:22	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN ...	0 row(s) affected
95	22:04:22	-- Execute Step 11 CALL GetCustomerAccounts(101)	1 row(s) returned
96	22:04:22	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount...	0 row(s) affected

Automatic content
Use the toolbar
help for the current
or to toggle

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

```

120      IN p_Account_No INT,
121      IN p_Amount DECIMAL(12,2)
122  )
123  BEGIN
124      UPDATE Account
125      SET Balance = Balance + p_Amount
126      WHERE Account_No = p_Account_No;
127  END //
128  DELIMITER ;

```

Result Grid Filter Rows: Export: Wrap Cell Content:

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Geethika	10001	Savings	50000.00	Hyderabad

Administration Schemas

Information

No object selected

Customer 13 Account 14 Bank_Transaction 15 Result 16 Result 17 Result 18 x

Output

Action Output

#	Time	Action	Message
91	22:04:22	-- Execute Step 9 CALL GetAllCustomers()	5 row(s) returned
92	22:04:22	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN ...	0 row(s) affected
93	22:04:22	-- Execute Step 10 CALL GetAccountDetails(10001)	1 row(s) returned
94	22:04:22	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN...	0 row(s) affected
95	22:04:22	-- Execute Step 11 CALL GetCustomerAccounts(101)	1 row(s) returned
96	22:04:22	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount...	0 row(s) affected

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Query 1 x

Limit to 1000 rows

```

136 BEGIN
137     UPDATE Account
138     SET Balance = Balance - p_Amount
139     WHERE Account_No = p_Account_No;
140 END //
141 DELIMITER ;
142 -- Execute Step 13
143 CALL WithdrawMoney(10001, 3000);
144

```

Result Grid

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

Administration Schemas

Information: No object selected

Customer 19 Account 20 Bank_Transaction 21 Result 22 Result 23 x Result 24 Read Only Context H

Output

Action Output

#	Time	Action	Message
97	22:06:40	DROP DATABASE IF EXISTS Triggers_DB	3 row(s) affected
98	22:06:40	CREATE DATABASE Triggers_DB	1 row(s) affected
99	22:06:40	USE Triggers_DB	0 row(s) affected
100	22:06:40	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name V...	0 row(s) affected
101	22:06:40	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID I...	0 row(s) affected
102	22:06:40	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUT...	0 row(s) affected
103	22:06:40	INSERT INTO Customer VALUES (101, 'Geethika', '9876543210', 'harshika@gmail...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

Object Info Session

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Query 1 x

Limit to 1000 rows

```

148 BEFORE UPDATE ON Account
149 FOR EACH ROW
150 BEGIN
151     IF NEW.Balance < 0 THEN
152         SIGNAL SQLSTATE '45000'
153         SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
154     END IF;
155 END //
156 DELIMITER ;

```

Result Grid

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Geethika	10001	Savings	50000.00	Hyderabad

Administration Schemas

Information

No object selected

Customer 19 Account 20 Bank_Transaction 21 Result 22 Result 23 Result 24 x

Read Only Context

Output

Action Output

#	Time	Action	Message
97	22:06:40	DROP DATABASE IF EXISTS Triggers_DB	3 row(s) affected
98	22:06:40	CREATE DATABASE Triggers_DB	1 row(s) affected
99	22:06:40	USE Triggers_DB	0 row(s) affected
100	22:06:40	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name V...	0 row(s) affected
101	22:06:40	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID I...	0 row(s) affected
102	22:06:40	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUT...	0 row(s) affected
103	22:06:40	INSERT INTO Customer VALUES (101, 'Geethika', '9876543210', harshika@gmail...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

Object Info Session

```

90 FROM Customer C
91 JOIN Account A
92 ON C.Customer_ID = A.Customer_ID
93 WHERE C.Customer_ID = p_Customer_ID;
94 END //
95
96 DELIMITER ;
97 -- Execute Step 11

```

Result Grid

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Harshika	10001	Savings	50000.00	Hyderabad

Result 9 x

Output

Action Output

#	Time	Action	Message
16	22:17:16	CALL GetAllCustomers()	5 row(s) returned
17	22:17:57	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
18	22:18:19	CALL GetAllCustomers()	5 row(s) returned
19	22:18:42	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
20	22:19:35	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
21	22:19:56	CALL GetCustomerAccounts(101)	1 row(s) returned

```

97 • -- Execute Step 11
98 CALL GetCustomerAccounts(101);
99 -- Step 12: Procedure to Deposit Money
100 DELIMITER //
101
102 • CREATE PROCEDURE DepositMoney(
103     IN p_Account_No INT,
104     IN p_Amount DECIMAL(12,2)
105 )
106 BEGIN
107     UPDATE Account
108     SET Balance = Balance + p_Amount
109     WHERE Account_No = p_Account_No;
110 END //
111
112 DELIMITER ;
113 • -- Execute Step 12
114 CALL DepositMoney(10001,5000);

```

Output

Action Output

#	Time	Action	Message
✓ 17	22:17:57	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account ...	0 row(s) affected
✓ 18	22:18:19	CALL GetAllCustomers()	5 row(s) returned
✓ 19	22:18:42	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
✓ 20	22:19:35	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
✓ 21	22:19:56	CALL GetCustomerAccounts(101)	1 row(s) returned
✓ 22	22:20:35	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected

```

102 • CREATE PROCEDURE DepositMoney(
103     IN p_Account_No INT,
104     IN p_Amount DECIMAL(12,2)
105 )
106 BEGIN
107     UPDATE Account
108     SET Balance = Balance + p_Amount
109     WHERE Account_No = p_Account_No;
110 END //
111
112 DELIMITER ;
113 • -- Execute Step 12
114 CALL DepositMoney(10001,5000);

```

Output

Action Output

#	Time	Action	Message
✓ 18	22:18:19	CALL GetAllCustomers()	5 row(s) returned
✓ 19	22:18:42	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
✓ 20	22:19:35	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
✓ 21	22:19:56	CALL GetCustomerAccounts(101)	1 row(s) returned
✓ 22	22:20:35	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 23	22:20:52	CALL DepositMoney(10001,5000)	1 row(s) affected

```

115  -- Step 13: Procedure to Withdraw Money
116  DELIMITER //
117
118  CREATE PROCEDURE WithdrawMoney(
119      IN p_Account_No INT,
120      IN p_Amount DECIMAL(12,2)
121  )
122  BEGIN
123      UPDATE Account
124      SET Balance = Balance - p_Amount
125      WHERE Account_No = p_Account_No;
126  END //
127
128  DELIMITER ;
129  -- Execute Step 13
130  CALL WithdrawMoney(10001,3000);

```

Output

Action Output

#	Time	Action	Message
✓ 19	22:18:42	SELECT * FROM Bank_Transaction LIMIT 0, 1000	5 row(s) returned
✓ 20	22:19:35	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
✓ 21	22:19:56	CALL GetCustomerAccounts(101)	1 row(s) returned
✓ 22	22:20:35	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 23	22:20:52	CALL DepositMoney(10001,5000)	1 row(s) affected
✓ 24	22:21:36	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected

```

117
118  CREATE PROCEDURE WithdrawMoney(
119      IN p_Account_No INT,
120      IN p_Amount DECIMAL(12,2)
121  )
122  BEGIN
123      UPDATE Account
124      SET Balance = Balance - p_Amount
125      WHERE Account_No = p_Account_No;
126  END //
127
128  DELIMITER ;
129  -- Execute Step 13
130  CALL WithdrawMoney(10001,3000);

```

Output

Action Output

#	Time	Action	Message
✓ 20	22:19:35	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, ...	0 row(s) affected
✓ 21	22:19:56	CALL GetCustomerAccounts(101)	1 row(s) returned
✓ 22	22:20:35	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 23	22:20:52	CALL DepositMoney(10001,5000)	1 row(s) affected
✓ 24	22:21:36	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 25	22:21:51	CALL WithdrawMoney(10001,3000)	1 row(s) affected

Step 3 : TRRIGERS

30485 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- aggregate_bank_db
- bank_management_db
- bookflow_db
- joins_operations_db
- physician_department_db
- std_marks
- sys

Query 1

Limit to 1000 rows

VALUES

```

194
195 (
196     NEW.Transaction_ID,
197     NEW.Account_No,
198     NEW.Transaction_Type,
199     NEW.Amount
200 );
201 END //
202 DELIMITER ;

```

Result Grid

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Geethika	10001	Savings	50000.00	Hyderabad

Administration Schemas

Information

No object selected

Customer 25 Account 26 Bank_Transaction 27 Result 28 Result 29 Result 30 x

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
134	22:10:27	-- Execute Step 10 CALL GetAccountDetails(10001)	1 row(s) returned
135	22:10:27	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN...	0 row(s) affected
136	22:10:27	-- Execute Step 11 CALL GetCustomerAccounts(101)	1 row(s) returned
137	22:10:27	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount...	0 row(s) affected
138	22:10:27	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount...	0 row(s) affected
139	22:10:27	-- Execute Step 13 CALL WithdrawMoney(10001, 3000)	1 row(s) affected
140	22:10:27	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH R...	0 row(s) affected

Object Info Session

```

145 -- Step 15: Trigger to Prevent Invalid Transaction Amount
146 DELIMITER //
147
148 CREATE TRIGGER CheckTransactionAmount
149 BEFORE INSERT ON Bank_Transaction
150 FOR EACH ROW
151 BEGIN
152     IF NEW.Amount <= 0 THEN
153         SIGNAL SQLSTATE '45000'
154         SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
155     END IF;
156 END //
157
158 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
22	22:20:35	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
23	22:20:52	CALL DepositMoney(10001,5000)	1 row(s) affected
24	22:21:36	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
25	22:21:51	CALL WithdrawMoney(10001,3000)	1 row(s) affected
26	22:23:13	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected


```

159 • -- Step 16: Create Transaction Audit Table
160 CREATE TABLE Transaction_Audit (
161     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
162     Transaction_ID INT,
163     Account_No INT,
164     Transaction_Type VARCHAR(20),
165     Amount DECIMAL(12,2),
166     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
167 );

```

Output

Action Output

#	Time	Action	Message
✓ 23	22:20:52	CALL DepositMoney(10001,5000)	1 row(s) affected
✓ 24	22:21:36	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 25	22:21:51	CALL WithdrawMoney(10001,3000)	1 row(s) affected
✓ 26	22:23:13	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
✓ 27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected

```

168 -- Step 17: Trigger for Transaction Audit
169 DELIMITER //
170
171 • CREATE TRIGGER TransactionAudit
172 AFTER INSERT ON Bank_Transaction
173 FOR EACH ROW
174 BEGIN
175     INSERT INTO Transaction_Audit
176     (Transaction_ID,Account_No,Transaction_Type,Amount)
177     VALUES
178     (NEW.Transaction_ID,NEW.Account_No,NEW.Transaction_Type,NEW.Amount);
179 END //
180
181 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
✓ 24	22:21:36	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 25	22:21:51	CALL WithdrawMoney(10001,3000)	1 row(s) affected
✓ 26	22:23:13	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
✓ 27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected

```

183 DELIMITER //
184
185 • CREATE TRIGGER UpdateBalanceAfterTransaction
186 AFTER INSERT ON Bank_Transaction
187 FOR EACH ROW
188 BEGIN
189     IF NEW.Transaction_Type = 'DEPOSIT' THEN
190         UPDATE Account
191         SET Balance = Balance + NEW.Amount
192         WHERE Account_No = NEW.Account_No;
193
194     ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
195         UPDATE Account
196         SET Balance = Balance - NEW.Amount
197         WHERE Account_No = NEW.Account_No;
198     END IF;
199 END //
200
201 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
✓ 25	22:21:51	CALL WithdrawMoney(10001,3000)	1 row(s) affected
✓ 26	22:23:13	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
✓ 27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
✓ 30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected

```

202  -- Step 19: Trigger to Prevent Insufficient Withdrawal
203  DELIMITER //
204
205  • CREATE TRIGGER PreventInsufficientWithdrawal
206  BEFORE INSERT ON Bank_Transaction
207  FOR EACH ROW
208  BEGIN
209      DECLARE CurrentBalance DECIMAL(12,2);
210
211      SELECT Balance
212      INTO CurrentBalance
213      FROM Account
214      WHERE Account_No = NEW.Account_No;
215
216      IF NEW.Transaction_Type = 'WITHDRAW'
217      AND NEW.Amount > CurrentBalance THEN
218          SIGNAL SQLSTATE '45000'
219          SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
220      END IF;
221  END //
222
223  DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
✓ 26	22:23:13	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
✓ 27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
✓ 30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected

Step 4 : ADVANCED PROCEDURES

```

224  -- Step 20: Procedure for Money Transfer
225  DELIMITER //
226
227  CREATE PROCEDURE TransferMoney(
228      IN SenderAccount INT,
229      IN ReceiverAccount INT,
230      IN TransferAmount DECIMAL(12,2)
231  )
232  BEGIN
233      DECLARE SenderBalance DECIMAL(12,2);
234
235      SELECT Balance
236      INTO SenderBalance
237      FROM Account
238      WHERE Account_No = SenderAccount;
239
240      IF SenderBalance < TransferAmount THEN
241          SIGNAL SQLSTATE '45000'
242          SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
243      ELSE
244          UPDATE Account
245          SET Balance = Balance - TransferAmount

```

Output

Action Output

#	Time	Action	Message
✓ 27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
✓ 30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected

```

237     FROM Account
238     WHERE Account_No = SenderAccount;
239
240     IF SenderBalance < TransferAmount THEN
241         SIGNAL SQLSTATE '45000'
242         SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
243     ELSE
244         UPDATE Account
245         SET Balance = Balance - TransferAmount
246         WHERE Account_No = SenderAccount;
247
248         UPDATE Account
249         SET Balance = Balance + TransferAmount
250         WHERE Account_No = ReceiverAccount;
251     END IF;
252 END //
253
254 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
✓ 27	22:23:45	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
✓ 30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected


```

254 DELIMITER ;
255 • -- Execute Step 20
256 CALL TransferMoney(10001,10002,5000);
257 -- Step 21: Procedure for High Balance Accounts
258 DELIMITER //
259
260 • CREATE PROCEDURE HighBalanceAccounts(
261     IN MinimumBalance DECIMAL(12,2)
262 )
263 BEGIN
264     SELECT *
265     FROM Account
266     WHERE Balance >= MinimumBalance
267     ORDER BY Balance DESC;
268 END //
269
270 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
✓ 28	22:24:59	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
✓ 30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
✓ 33	22:29:07	CALL TransferMoney(10001,10002,5000)	1 row(s) affected

```

257 -- Step 21: Procedure for High Balance Accounts
258 DELIMITER //
259
260 • CREATE PROCEDURE HighBalanceAccounts(
261     IN MinimumBalance DECIMAL(12,2)
262 )
263 BEGIN
264     SELECT *
265     FROM Account
266     WHERE Balance >= MinimumBalance
267     ORDER BY Balance DESC;
268 END //
269
270 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
✓ 29	22:25:33	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
✓ 30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
✓ 32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
✓ 33	22:29:07	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
✓ 34	22:29:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected

```

268 END //
269
270 DELIMITER ;
271 -- Execute Step 21

```

Result Grid		Filter Rows:		Export:	Wrap Cell Content:
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10002	102	Savings	80000.00	Vijayawada

Result 10 x

Output

Action Output

#	Time	Action	Message
30	22:26:31	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
33	22:29:07	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
34	22:29:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
35	22:30:04	CALL HighBalanceAccounts(50000)	3 row(s) returned

```

275 CREATE PROCEDURE GetBalance(
276     IN p_Account_No INT,
277     OUT p_Balance DECIMAL(12,2)
278 )
279 BEGIN
280     SELECT Balance
281     INTO p_Balance
282     FROM Account
283     WHERE Account_No = p_Account_No;
284 END //
285
286 DELIMITER ;
287 CALL HighBalanceAccounts(50000);

```

Output

Action Output

#	Time	Action	Message
31	22:27:17	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
33	22:29:07	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
34	22:29:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
35	22:30:04	CALL HighBalanceAccounts(50000)	3 row(s) returned
36	22:30:37	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected

```

283     WHERE Account_No = p_Account_No;
284 END //
285
286 DELIMITER ;
287 • CALL HighBalanceAccounts(50000);

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10002	102	Savings	80000.00	Vijayawada

Result 11 x

Output

 Action Output 

	#	Time	Action	Message
✓	32	22:28:21	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
✓	33	22:29:07	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
✓	34	22:29:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
✓	35	22:30:04	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓	36	22:30:37	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓	37	22:30:53	CALL HighBalanceAccounts(50000)	3 row(s) returned

287

```

286 DELIMITER ;
287 • CALL HighBalanceAccounts(50000);
288 -- Execute Step 22
289 • CALL GetBalance(10001,@CurrentBalance);
290
291 • SELECT @CurrentBalance;

```

Output

 Action Output 

	#	Time	Action	Message
✓	33	22:29:07	CALL TransferMoney(10001,10002,5000)	1 row(s) affected
✓	34	22:29:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
✓	35	22:30:04	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓	36	22:30:37	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓	37	22:30:53	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓	38	22:32:00	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected

```

288      -- Execute Step 22
289      CALL GetBalance(10001,@CurrentBalance);
290
291      SELECT @CurrentBalance;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	@CurrentBalance
▶	47000.00

Result 12 x

Output

Action Output

#	Time	Action	Message
✓ 34	22:29:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
✓ 35	22:30:04	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓ 36	22:30:37	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 37	22:30:53	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓ 38	22:32:00	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected
✓ 39	22:32:16	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned

Step 5 : Management

```

291      SELECT @CurrentBalance;
292      -- Step 23: Display All Triggers
293      SHOW TRIGGERS;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Trigger	Event	Table	Statement	Timing	Created	sql_mode
▶ CheckBalance	UPDATE	account	BEGIN IF NEW.Balance < 0 THEN SIGNA...	BEFORE	2026-08-13 22:23:13.56	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE
CheckTransactionAmount	INSERT	bank_transaction	BEGIN IF NEW.Amount <= 0 THEN SIGN...	BEFORE	2026-08-13 22:23:45.18	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE
PreventInsufficientWithdrawal	INSERT	bank_transaction	BEGIN DECLARE CurrentBalance DECIMAL(12...	BEFORE	2026-08-13 22:27:17.93	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE
TransactionAudit	INSERT	bank_transaction	BEGIN INSERT INTO Transaction_Audit (Tr...	AFTER	2026-08-13 22:25:33.94	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE
UpdateBalanceAfterTransaction	INSERT	bank_transaction	BEGIN IF NEW.Transaction_Type = 'DEPOSIT...	AFTER	2026-08-13 22:26:31.76	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE

Result 13 x Read Only

Output

Action Output

#	Time	Action	Message
✓ 35	22:30:04	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓ 36	22:30:37	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 37	22:30:53	CALL HighBalanceAccounts(50000)	3 row(s) returned
✓ 38	22:32:00	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected
✓ 39	22:32:16	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
✓ 40	22:32:54	SHOW TRIGGERS	5 row(s) returned

294 -- Step 24: Display Trigger Definition

295 • SHOW CREATE TRIGGER CheckBalance;

Result Grid Filter Rows: Export: Wrap Cell Content: I

Trigger	sql_mode	SQL Original Statement	character_set_client	collation_connection	Database Collation	Created
CheckBalance	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER='root'@'localhost' TRIGGE...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci	2026-08-13 22

Result 14 x

Read Only

Output

Action Output

#	Time	Action	Message
36	22:30:37	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
37	22:30:53	CALL HighBalanceAccounts(50000)	3 row(s) returned
38	22:32:00	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected
39	22:32:16	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
40	22:32:54	SHOW TRIGGERS	5 row(s) returned
41	22:33:25	SHOW CREATE TRIGGER CheckBalance	1 row(s) returned

294 -- Step 24: Display Trigger Definition

295 • SHOW CREATE TRIGGER CheckBalance;

296 -- Step 25: Display All Procedures

297 • SHOW PROCEDURE STATUS

Result Grid Filter Rows: Export: Wrap Cell Content: I

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
----	------	------	---------	----------	---------	---------------	---------	----------------------	----------------------	--------------------

Result 15 x

Output

Action Output

#	Time	Action	Message
37	22:30:53	CALL HighBalanceAccounts(50000)	3 row(s) returned
38	22:32:00	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected
39	22:32:16	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
40	22:32:54	SHOW TRIGGERS	5 row(s) returned
41	22:33:25	SHOW CREATE TRIGGER CheckBalance	1 row(s) returned
42	22:33:55	SHOW PROCEDURE STATUS WHERE Db = 'Bank_Triggers_DB'	0 row(s) returned

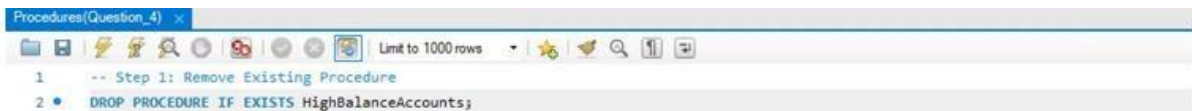
CONCEPT : PROCEDURES

QUESTION 4:

Create a procedure to display accounts whose balance is greater than a given amount.

Example:

CALL HighBalanceAccounts(100000);



```
Procedures(Question_4) x
Limit to 1000 rows

1  -- Step 1: Remove Existing Procedure
2  DROP PROCEDURE IF EXISTS HighBalanceAccounts;
3  -- Step 2: Create Procedure for High Balance Accounts
4  DELIMITER //
5
6  CREATE PROCEDURE HighBalanceAccounts(
7      IN MinimumBalance DECIMAL(12,2)
8  )
9  BEGIN
10     SELECT *
11     FROM Account
12     WHERE Balance > MinimumBalance;
13 END //
14
15 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
1	21:45:46	DROP PROCEDURE IF EXISTS HighBalanceAccounts	0 row(s) affected
2	21:46:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected

Procedures(Question_4) x

Limit to 1000 rows

```
9  BEGIN
10  SELECT *
11  FROM Account
12  WHERE Balance > MinimumBalance;
13  END //
14
15  DELIMITER ;
16  -- Step 3: Execute Procedure
17  CALL HighBalanceAccounts(100000);
```

Result Grid

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore

Result 1 x

Output

Action Output

#	Time	Action	Message
1	21:45:46	DROP PROCEDURE IF EXISTS HighBalanceAccounts	0 row(s) affected
2	21:46:21	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
3	21:46:40	CALL HighBalanceAccounts(100000)	1 row(s) returned

CONCEPT : TRIGGERS

QUESTION 5

Create a trigger that prevents an account balance from becoming negative.

```
Triggers(Question 5)
1 -- Step 1: Remove Existing Trigger
2 • DROP TRIGGER IF EXISTS CheckBalance;
```

Output			
Action Output			
#	Time	Action	Message
1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected

```
Triggers(Question 5)
1 -- Step 1: Remove Existing Trigger
2 • DROP TRIGGER IF EXISTS CheckBalance;
3 -- Step 2: Create Trigger to Prevent Negative Balance
4 DELIMITER //
5
6 • CREATE TRIGGER CheckBalance
7 BEFORE UPDATE ON Account
8 FOR EACH ROW
9 BEGIN
10 IF NEW.Balance < 0 THEN
11 SIGNAL SQLSTATE '45000'
12 SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
13 END IF;
14 END //
15
16 DELIMITER ;
```

Output			
Action Output			
#	Time	Action	Message
1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
2	21:52:49	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balanc...	0 row(s) affected

```

Triggers(Question 5)
1  -- Step 1: Remove Existing Trigger
2  DROP TRIGGER IF EXISTS CheckBalance;
3  -- Step 2: Create Trigger to Prevent Negative Balance
4  DELIMITER //
5
6  CREATE TRIGGER CheckBalance
7  BEFORE UPDATE ON Account
8  FOR EACH ROW
9  BEGIN
10     IF NEW.Balance < 0 THEN
11         SIGNAL SQLSTATE '45000'
12         SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
13     END IF;
14 END //
15
16 DELIMITER ;
17 -- Step 3: Test Trigger
18 UPDATE Account
19 SET Balance = -1000
20 WHERE Account_No = 10001;

```

Output			
Action Output			
#	Time	Action	Message
✓ 1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
✓ 2	21:52:49	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balanc...	0 row(s) affected
✗ 3	21:53:16	UPDATE Account SET Balance = -1000 WHERE Account_No = 10001	Error Code: 1644. Transaction failed: Insufficient balance

```

Triggers(Question 5)
16 DELIMITER ;
17 -- Step 3: Test Trigger
18 UPDATE Account
19 SET Balance = -1000
20 WHERE Account_No = 10001;
21 -- Step 4: Verify Account Balance
22 SELECT *
23 FROM Account
24 WHERE Account_No = 10001;

```

Result Grid					
Filter Rows:					
Edit: Export/Import: Wrap Cell Content: I					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	47000.00	Hyderabad
*	NULL	NULL	NULL	NULL	NULL

Account 1			
Output			
Action Output			
#	Time	Action	Message
✓ 1	21:52:25	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
✓ 2	21:52:49	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balan...	0 row(s) affected
✗ 3	21:53:16	UPDATE Account SET Balance = -1000 WHERE Account_No = 10001	Error Code: 1644. Transaction failed: Insufficient balance
✓ 4	21:53:36	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	1 row(s) returned

QUESTION 6

Create a trigger that prevents a transaction amount from being zero or negative.

```
Triggers(Question 6) x
Limit to 1000 rows
1 -- Step 1: Remove Existing Trigger
2 • DROP TRIGGER IF EXISTS CheckTransactionAmount;
```

Output

Action Output

#	Time	Action	Message
1	21:55:08	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected

```
Triggers(Question 6) x
Limit to 1000 rows
1 -- Step 1: Remove Existing Trigger
2 • DROP TRIGGER IF EXISTS CheckTransactionAmount;
3 -- Step 2: Create Trigger to Prevent Invalid Transaction Amount
4 DELIMITER //
5
6 • CREATE TRIGGER CheckTransactionAmount
7 BEFORE INSERT ON Bank_Transaction
8 FOR EACH ROW
9 BEGIN
10 IF NEW.Amount <= 0 THEN
11 SIGNAL SQLSTATE '45000'
12 SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
13 END IF;
14 END //
15
16 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
1	21:55:08	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
2	21:55:27	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGI...	0 row(s) affected

Triggers(Question 6)

```

2 • DROP TRIGGER IF EXISTS CheckTransactionAmount;
3 -- Step 2: Create Trigger to Prevent Invalid Transaction Amount
4 DELIMITER //
5
6 • CREATE TRIGGER CheckTransactionAmount
7 BEFORE INSERT ON Bank_Transaction
8 FOR EACH ROW
9 BEGIN
10 IF NEW.Amount <= 0 THEN
11     SIGNAL SQLSTATE '45000'
12     SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
13 END IF;
14 END //
15
16 DELIMITER ;
17 • -- Step 3: Test Zero Transaction Amount
18 INSERT INTO Bank_Transaction
19 (Account_No, Transaction_Type, Amount)
20 VALUES
21 (10001, 'DEPOSIT', 0);

```

Output

Action Output

#	Time	Action	Message
✓ 1	21:55:08	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
✓ 2	21:55:27	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGI...	0 row(s) affected
✗ 3	21:55:48	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 0)	Error Code: 1644. Transaction amount must be greater than zero

Triggers(Question 6)

```

7 BEFORE INSERT ON Bank_Transaction
8 FOR EACH ROW
9 BEGIN
10 IF NEW.Amount <= 0 THEN
11     SIGNAL SQLSTATE '45000'
12     SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
13 END IF;
14 END //
15
16 DELIMITER ;
17 • -- Step 3: Test Zero Transaction Amount
18 INSERT INTO Bank_Transaction
19 (Account_No, Transaction_Type, Amount)
20 VALUES
21 (10001, 'DEPOSIT', 0);
22 -- Step 4: Test Negative Transaction Amount
23 • INSERT INTO Bank_Transaction
24 (Account_No, Transaction_Type, Amount)
25 VALUES
26 (10001, 'DEPOSIT', -5000);

```

Output

Action Output

#	Time	Action	Message
✓ 1	21:55:08	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
✓ 2	21:55:27	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGI...	0 row(s) affected
✗ 3	21:55:48	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 0)	Error Code: 1644. Transaction amount must be greater than zero
✗ 4	21:56:56	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', -5000)	Error Code: 1644. Transaction amount must be greater than zero